

Universidade do Minho

Escola de Engenharia

Mestrado em Engenharia de
Telecomunicações e Informática
Escola de Engenharia

Relatório de Especificação da Fase B
Projeto Integrador em Telecomunicações e Informática

Protótipo De Um Sistema De Entretenimento (e utilitário) Num Aeroporto

Grupo 3 constituído por:

Luís Pedro Cerqueira Baptista, PG61434

Bernardo Gualdino Salgado, PG59972

Nuno Duarte Mirra, PG57594

Pedro João Daniel, PG58741

2025/2026

Índice

1. Introdução	4
2. Arquitetura do Sistema e Especificações	4
2.2 Escolha do Algoritmo de Verificação de Erros	8
2.3 Escolha do Mecanismo de Correção de Erros	9
2.4 Cálculos - Tempo de Transmissão de um Ficheiro	11
3. Simulação e Avaliação de Desempenho	14
3.1 Testes e Validação do Sistema	16
4. Planeamento, Competências e Recursos.....	16
4.1 Competências Técnicas	16
4.2 Planeamento de Tarefas	17
5. Conclusão	19

Índice de Figuras

Figura 1 - Diagrama de Blocos	5
Figura 2 - Estrutura da Trama	6
Figura 3 - Fluxograma da máquina de estados do protocolo Stop-and-Wait.....	11
Figura 4 - Diagrama de sequência temporal dos cenários de transmissão e recuperação de falhas	15
Figura 5 - Diagrama Temporal (Gant)	19

1. Introdução

Na fase B, passamos da camada física (Fase A) para a camada de ligação de dados (Camada 2 do Modelo OSI). O objetivo principal é garantir uma comunicação ponto-a-ponto, full-duplex e fiável entre dois computadores através das placas ESP32.

Para isso, teremos de implementar o protocolo de acesso ao meio (MAC), aplicando o controlo de ligação lógica (LLC) de forma a agrupar os bits “brutos” em tramas e possibilitar o acesso ao meio sem colisões, evitando perdas de bytes da trama, perdas de tramas ou troca de valores de bits na trama. O sistema da fase B deverá efetuar a transferência de ficheiros e textos, garantindo que não há perdas de informação.

2. Arquitetura do Sistema e Especificações

A arquitetura da camada 2 (representada na **Figura 1**) representa o limite das tramas, a deteção e controlo de erros e definição de APIs. Note-se que a porta série da ESP32 operará no modo raw a 9600 baud rate.

O digrama de blocos da Figura 1 é a integração da nossa camada de ligação (Fase B) com as outras camadas do projeto (Física/Aplicação).

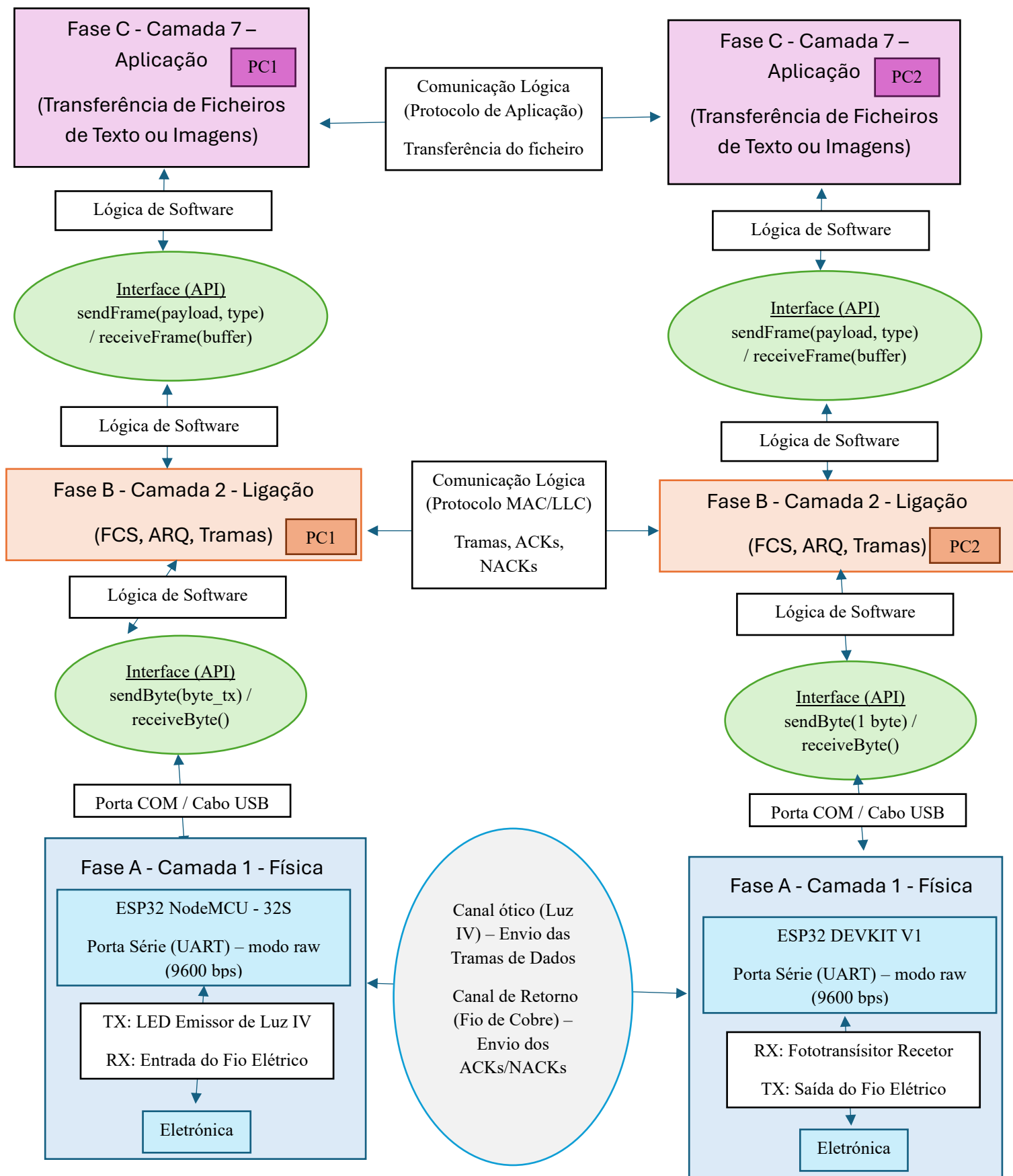


Figura 1 - Diagrama de Blocos

2.1 Estrutura da Trama

A camada física (Fase A), limita-se a transmitir e receber um fluxo contínuo e não estruturado de bits (0's e 1's) através do feixe de luz. Contudo, este meio não oferece qualquer informação sobre onde começa ou acabada uma mensagem, nem garante que os bits não sofram alterações. Por essa mesma razão, que a camada da ligação de dados estabelece a definição de uma trama. O encapsulamento dos dados brutos numa trama permite-nos:

- Sincronização e delimitação
 - Permite ao recetor identificar o início e o fim de um pacote de dados, extraíndo a informação útil.
- Detecção de erros
 - Ao aplicar um algoritmo de verificação de erros a um bloco delimitado, é possível garantir a integridade daquela secção específica de dados.
- Controlo de fluxo e recuperação de tramas perdidas
 - Esta encapsulação dos dados brutos numa trama, permite implementar mecanismos de retransmissão, garantindo que em caso de erro ou perda, apenas a trama afetada seja reenviada, e não a totalidade do ficheiro.
- Endereçamento
 - Permite identificar a estação emissora e recetora envolvidas na comunicação.

A estrutura da trama foi definida com os seguintes campos na **Figura 2**:

SOF (1byte)	Destination MAC (1 byte)	Source MAC (1 byte)	TYPE (1 byte):	SEQ_NUM (1 byte)	Length (1 byte)	PAYLOAD $256 \geq N \geq 68$ bytes	FCS (1 ou 2 bytes)
----------------	--------------------------------	---------------------------	-------------------	---------------------	--------------------	---------------------------------------	-----------------------

Figura 2 - Estrutura da Trama

- Delimitador de Início (Start of Frame – SOF)
 - Caractere especial de controlo responsável pela delimitação do início da trama. Permite ao recetor sincronizar-se e identificar quando começa um novo pacote de dados.
- Endereço de Destino (Destination MAC)
 - Identifica o dispositivo que irá receber o pacote de dados. Garante que a trama é processada apenas pelo dispositivo a que se destina.
- Endereço de Origem (Source MAC)

- Identifica o dispositivo que envia o pacote de dados. Permite ao recetor saber quem originou a mensagem, sendo por isso, essencial para o envio das mensagens de confirmação (ACK).
- Tipo de Trama (Type)
 - Campo de controlo que desempenha duas funcionalidades. Em primeiro lugar, é essencial para distinguir tramas de dados, que transportam a informação real do ficheiro, de tramas de controlo, que incluem as mensagens de confirmação de receção (ACKs). Em segundo lugar, atua como um mecanismo de multiplexagem, para a interface de aplicação. Uma vez que o sistema irá suportar diferentes serviços e tipos de dados em simultâneo, o campo Type indica à camada 2, a qual protocolo ou aplicação de camada superior deve ser entregue a informação útil contida na trama, após a validação da mesma.
 - Tramas de Controlo (Camada de Ligação):
 - 0x0A (ACK): Confirmação positiva. Indica que a trama anterior foi recebida com sucesso e com o FCS válido.
 - 0x0B (NACK): Confirmação negativa. Indica que a trama foi recebida, mas falhou a validação do FCS.
 - Tramas de Dados (Multiplexagem para a Camada de Aplicação):
 - 0x01: (**Protocolo A - TCP**): Indica que o *payload* se destina ao protocolo TCP (*Transmission Control Protocol*). Este identificador será associado à transferência de ficheiros de texto.
 - 0x02: (**Protocolo B - UDP**): Indica que o *payload* se destina ao protocolo UDP (*User Datagram Protocol*). Este identificador será associado à transferência de conteúdos multimédia.
- Número de Sequência (Sequence Number)
 - Identificador numérico atribuído sequencialmente a cada pacote enviado.

- Tamanho (Length)
 - Indica o número exato de bytes que o campo de dados contém. Permite ao recetor saber quando termina a informação útil e onde começa o bloco de verificação de erros.
- Informação útil (Payload)
 - Sequência de dados a transmitir (até N bytes, sendo $256 \geq N \geq 68$ bytes).
- Sequência de Verificação de Erros (FCS – Frame Check Sequence)
 - Campo que contém um valor calculado (como Checksum ou CRC), com base no conteúdo da trama. Destina-se ao controlo de erros, o que permite ao recetor validar a integridade dos dados e solicitar retransmissões em caso de corrupção de bits durante a transmissão.

2.2 Escolha do Algoritmo de Verificação de Erros

A escolha do algoritmo para a Sequência de Verificação de Erros (FCS) é determinante para garantir a fiabilidade da transmissão. Durante esta fase, foram ponderadas duas abordagens: o Checksum e o CRC (Cyclic Redundancy Check).

- Checksum
 - Esta estratégia consiste em somar todos os bytes na trama a enviar, sendo o resultado anexado ao pacote para validação no recetor. Contudo, esta técnica apresenta limitações, pois não consegue detetar a troca de posição de bytes nem a inserção de bytes com valor zero, assim sendo, a deteção de erros é muito fraca nesta técnica.
- CRC (Cyclic Redundancy Check)
 - Trata a mensagem como um longo polinómio binário e divide-o por um polinómio gerador fixo. O resto desta divisão é o valor do CRC anexado à trama. Por um lado, é matematicamente mais robusto, já que um CRC de 8 bits ou 16 bits, deteta 100% dos erros simples e duplos, e também erros em números ímpares de bits. Por outro lado, a sua implementação é mais complexa e exige maior poder de processamento se for calculado bit a bit.

Numa fase inicial, será implementado o algoritmo de Checksum devido à sua simplicidade. Numa fase posterior, pretende-se implementar o algoritmo de CRC-8 (8 bits), isto irá resolver as limitações apresentadas pelo algoritmo de Checksum, e melhorar a capacidade de deteção de erros, o que consequentemente, melhora a eficiência da transmissão com um baudrate de 9600 bps.

2.3 Escolha do Mecanismo de Correção de Erros

Após a deteção de erros na trama recebida, validada através do Checksum ou CRC, devemos proceder à correção desses mesmo erros, garantindo a entrega do ficheiro sem erros. Para atingir este objetivo, foram analisadas duas estratégias: o FEC (Forward Error Correction) e ARQ (Automatic Repeat reQuest).

- FEC

→ O emissor adiciona bits de redundância calculados matematicamente através do código de Hamming ou Reed-Solomon, permitindo ao recetor corrigir erros sem retransmissão. Evita atrasos, mas aumenta o tamanho das tramas e, por um lado, introduz um overhead considerável em todas as tramas, o que reduz a taxa de transferência e exige mais poder computacional.

- ARQ

→ Esta estratégia baseia-se no princípio de retransmissão. O recetor valida a trama, se esta estiver correta, então envia uma mensagem de confirmação positiva (ACK). Se detetar um erro, então descarta a trama e pode enviar uma confirmação negativa (NACK), ou pode ignorar e fazer com que o emissor dispare um temporizador e reenvie a trama original. Por um lado, é altamente eficiente em canais onde a taxa de erro não é constante, pois só gasta mais largura de banda quando um erro efetivamente ocorre. Por outro lado, exige obrigatoriamente, um canal bidirecional, para permitir o envio de ACKs e NACKs, e introduz um atraso sempre que é necessária uma retransmissão.

A escolha da estratégia que iremos utilizar para a correção de erros, será o mecanismo ARQ. Isto porque é exigido que o sistema seja capaz de efetuar timeout em caso de perda de tramas e recuperar a transmissão exatamente no ponto onde parou.

O mecanismo de ARQ com temporizadores, resolve o problema dos bits corrompidos e também o problema das tramas totalmente perdidas. Para além disso, como exigido que implementemos no sistema uma comunicação full-duplex, este mecanismo garante a troca bidirecional de mensagens de controlo (ACKs e NACKs).

Tendo sido definida a estratégia ARQ para a correção dos erros, é necessário também escolher que variante do protocolo iremos escolher para implementar. Temos ao nosso dispor dois protocolos possíveis, o Stop-and-Wait e o Go-Back-N, que é baseada no conceito de janela deslizante.

- Stop-and-Wait

→ O emissor transmite uma única trama e suspende a transmissão ao iniciar um temporizador. O recetor avalia a trama, se esta for válida, envia uma mensagem de confirmação (ACK), e logo de seguida, o emissor transmite a próxima trama. No caso em que a trama não for válida, devido a bits corrompidos, ou se a trama se perder devido a uma interrupção no feixe ótico, o recetor ignora-a. O temporizador inicializado anteriormente no emissor, acabará por expirar, desencadeando a retransmissão automática dessa mesma trama.

- Go-Back-N (Janela Deslizante)

→ O emissor é autorizado a transmitir múltiplas tramas consecutivamente até ao limite da janela sem ser necessário a confirmação individual de cada uma. Isto possibilita ao recetor enviar ACKs acumulativos. No caso da trama 2 não ser válida ou se essa mesma trama se perder, o recetor descarta as tramas seguintes, por estarem fora de sequência. Quando o temporizador da trama 2 expira, o emissor recua N passos, e retransmite a trama 2 e todas as subsequentes na sua janela.

Inicialmente irá ser implementado o protocolo Stop-and-Wait, já que este protocolo é mais simples de implementar, e garante minimamente o requisito de efetuar timeout em caso de perda de tramas e de recuperar a transmissão exatamente no ponto em que parou.

O protocolo Go-Back-N teoricamente iria garantir um menor tempo de transmissão do ficheiro, mas como o tempo de propagação do sinal é praticamente nulo, devido ao facto de o link ótico ter apenas um alcance de 1 metro, a otimização de tempo oferecida pelo Go-Back-N é marginal, e, portanto, não justifica o envio de múltiplas tramas em simultâneo.

Mesmo assim para efeitos de comparação dos resultados e para comprovar a justificação de não utilizar este protocolo, irá se calcular o tempo de transmissão de um ficheiro para o protocolo Go-Back-N e para o protocolo Stop-and-Wait.

A lógica de decisão e a máquina de estados implementada para o protocolo Stop-and-Wait encontram-se ilustradas na **Figura 3**. Este fluxograma demonstra a mecânica de validação da Sequência de Verificação de Erros (FCS) do lado do recetor, bem como o disparo do temporizador (timeout) e consequente retransmissão do lado do emissor

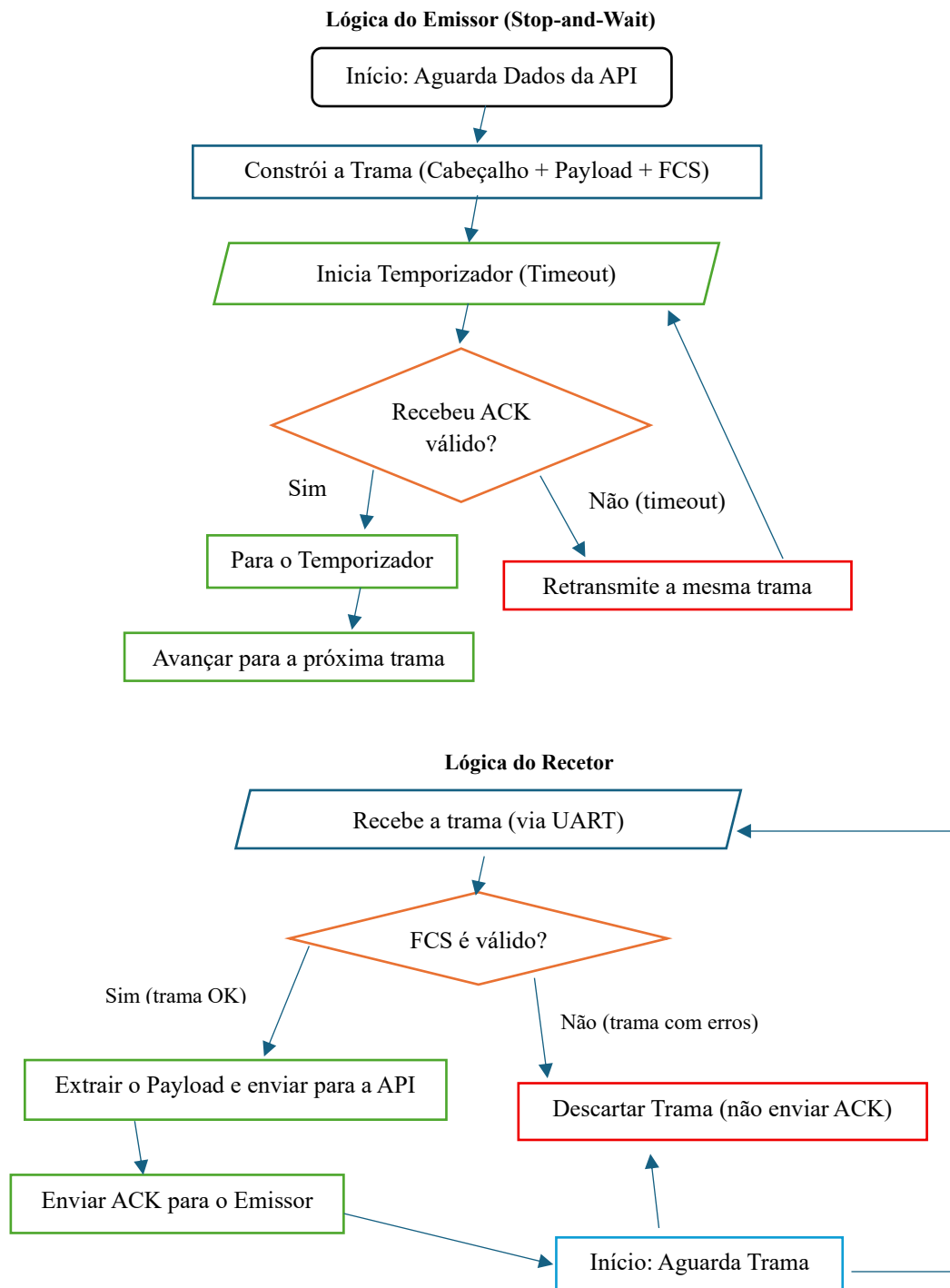


Figura 3 - Fluxograma da máquina de estados do protocolo Stop-and-Wait

2.4 Cálculos - Tempo de Transmissão de um Ficheiro

2.4.1 Protocolo Stop-And-Wait

Definição dos Parâmetros do Sistema:

- Baudrate (R): 9600 bps

- Formato da Porta Série (UART): Configuração 8-N-1 (1 start bit, 8 bits de dados, sem paridade, 1 stop bit). Isto significa que cada byte transmitido no canal, transmite na realidade 10 bits.
- Tamanho do Ficheiro (S): 1024 bytes (1 KB).
- Informação Útil / Payload (P): 64 bytes.
- Tamanho do Cabeçalho (H): 6 bytes (SOF, Destination MAC, Source MAC, Type, Sequence Number, Length)
- Tamanho da Cauda (T): 1 byte (Checksum ou CRC-8).

Com o protocolo Stop-and-Wait que definimos, temos dois tipos de tramas a ser transmitidas no canal, as tramas de dados e as tramas de ACK.

- Tamanho da Trama de Dados: Cabeçalho + Payload + Cauda

$$L_{\text{dados}} = 6 + 64 + 1 = 71 \text{ bytes}$$

- Tamanho da Trama ACK: Como é apenas uma mensagem de confirmação, não leva dados do ficheiro. (Cabeçalho + Cauda)

$$L_{\text{ACK}} = 6 + 0 + 1 = 7 \text{ bytes}$$

Como a UART envia 10 bits por cada byte, temos de calcular o tempo que demora a enviar cada pacote no canal a 9600 bps.

- Tempo de envio da Trama de Dados t_{dados}

$$t_{\text{dados}} = \frac{L_{\text{dados}} \times 10}{R} \approx 0.074 \text{ s (74ms)}$$

- Tempo de envio da Trama ACK t_{ACK}

$$t_{\text{ACK}} = \frac{L_{\text{ACK}} \times 10}{R} \approx 0.0073 \text{ s (7.3 ms)}$$

No protocolo Stop-and-Wait, por cada trama de dados enviada, o emissor tem de esperar pelo respetivo ACK antes de enviar a próxima trama. Portanto, o tempo de um ciclo de transmissão é a soma dos dois tempos.

- Tempo de um Ciclo t_{ciclo}

$$t_{\text{ciclo}} = t_{\text{dados}} + t_{\text{ACK}} = 81.3 \text{ ms}$$

Para além disso, precisamos de saber quantas tramas precisamos enviar para enviar o ficheiro inteiro de 1024 bytes.

- Número de Tramas N_{tramas}

$$N_{\text{tramas}} = \frac{S}{P} = \frac{1024}{64} = 16 \text{ tramas}$$

Finalmente com estes valores conseguimos calcular o tempo total para enviar o ficheiro de 1 KB.

- Tempo Total t_{total}

$$t_{total} = N_{tramas} \times t_{ciclo} = 16 \times 81.3 \text{ ms} \approx 1300 \text{ ms (1.3 segundos)}$$

Concluimos que o tempo teórico mínimo para transmitir um ficheiro de texto de 1 KB a 9600 bps, utilizando o protocolo Stop-And-Wait e com o formato da trama desenhado anteriormente, é de aproximadamente 1.3 segundos.

2.4.2 Protocolo Go-Back-N

Primeiramente, calculamos o tamanho ideal da janela (W), esta janela define quantas tramas o emissor pode enviar, antes de ser obrigado a parar e a esperar por um ACK. Para que o canal seja utilizado a 100%, o tempo necessário para enviar a janela completa deve ser maior ou igual ao tempo de ida e de volta da primeira trama mais a sua confirmação.

$$W \geq \frac{t_{dados} + t_{ACK} + 2 \times t_{prop}}{t_{dados}}$$

Substituindo pelos valores do nosso sistema, e assumindo que o tempo de propagação é 0, devido à curta distância do link ótico:

$$W \geq \frac{74ms + 7.3ms}{74ms} = 1.1$$

Teoricamente, como o tempo de propagação é nulo, e o ACK é muito mais pequeno que a trama de dados, uma janela de 2, já seria suficiente para que o canal seja utilizado a 100%. Contudo, na prática, os microcontroladores ESP32, introduzem atrasos de processamento. Portanto definimos um tamanho de janela de $W = 4$.

Assumindo um cenário ideal em que não há perdas de tramas e a janela é suficientemente grande, o emissor transmite as 16 tramas de dados de forma continua.

$$t_{total} = (N_{tramas} \times t_{dados}) + t_{ACK} = (16 \times 74ms) + 7.3ms = 1.19 \text{ segundos}$$

Como podemos observar o protocolo Go-Back-N, diminui de facto, o tempo de transmissão do ficheiro, mas é apenas uma diferença marginal, de cerca de 10%. Deste modo, como a diferença não é significativa e o protocolo Go-Back-N adiciona uma complexidade adicional de software pela gestão de buffers e janelas deslizantes, não se justifica a escolha deste protocolo.

O Stop-And-Wait garante o mesmo nível de fiabilidade, é substancialmente mais simples de implementar e cumpre o requisito de efetuar timeout em caso de perda de tramas e recuperar a transmissão exatamente no ponto onde parou.

3. Simulação e Avaliação de Desempenho

Para avaliar o desempenho da camada de ligação de dados concebida, serão realizados testes práticos de transferência de ficheiros de texto e de imagem (com tamanho “hard coded” de 1024 bytes) entre dois computadores. A comunicação será efetuada através da porta série das placas ESP32, operando no modo raw a 9600 baud rate.

A avaliação será sobre a eficiência do protocolo Stop-and-Wait perante cenários de falha. Serão também introduzidos erros no canal de transmissão para validar o seguinte:

- Deteção e Correção de Erros: Validar se os bits corrompidos são detetados através do campo FCS e solicitar a retransmissão via mecanismo ARQ.
- Recuperação de Perda de Tramas: Interrupção temporária do feixe ótico para forçar a perda total de pacotes, validando se o emissor efetua o timeout corretamente e retoma a transmissão no ponto exato em que parou.

Para garantir que o sistema cumpre o requisito vital de recuperar a transmissão fiável no ponto exato em que parou, os testes previstos incidirão sobre três cenários distintos de comunicação no canal ótico. O fluxo bidirecional de mensagens e o comportamento dos temporizadores nestes cenários estão detalhados na **Figura 4**:

- Cenário A: Transmissão ideal com sucesso.
- Cenário B: Perda total de trama devido a interrupção física do feixe ótico.
- Cenário C: Corrupção de bits validada e rejeitada pelo campo FCS

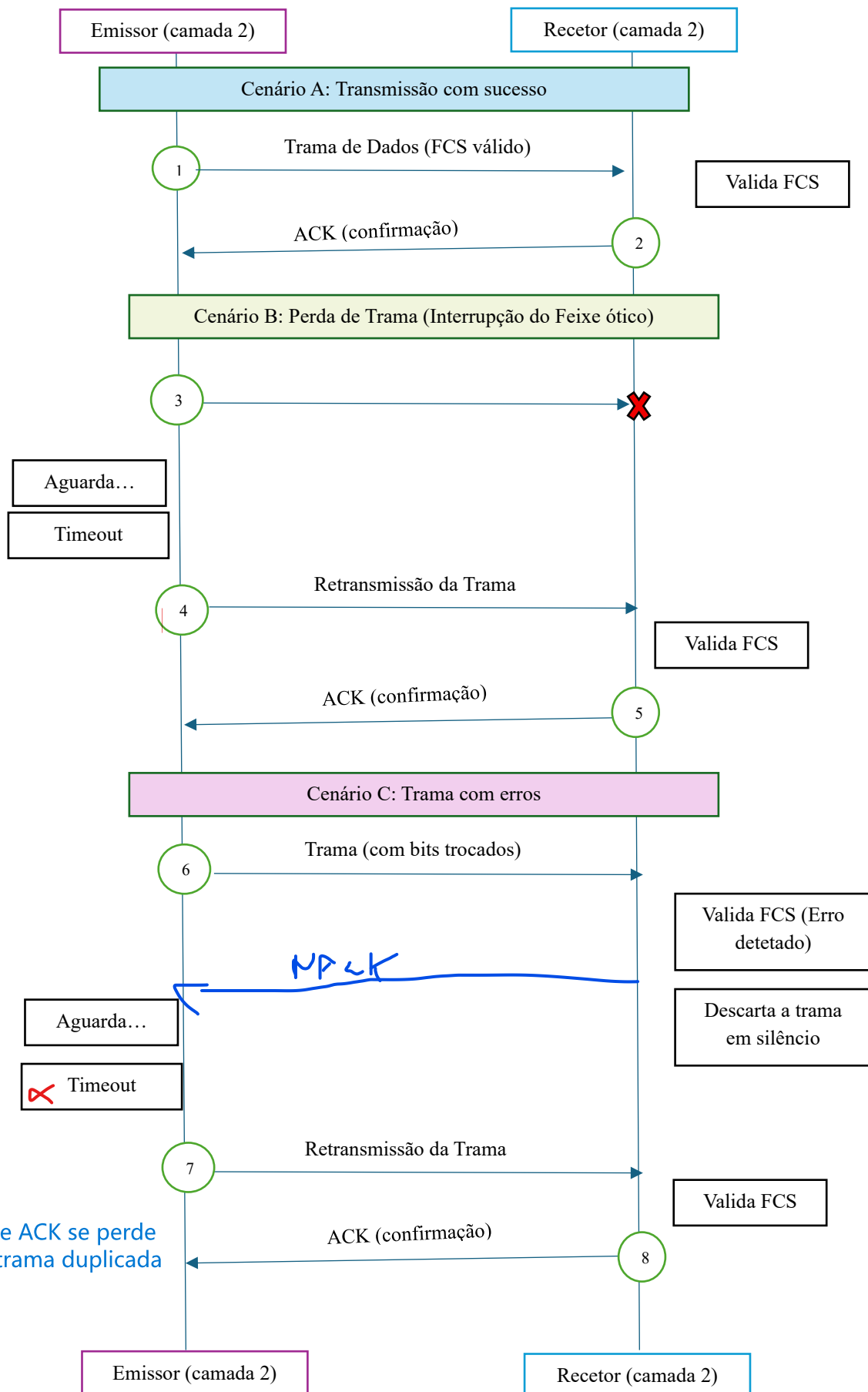


Figura 4 - Diagrama de sequência temporal dos cenários de transmissão e recuperação de falhas

3.1 Testes e Validação do Sistema

Para validar o sistema, serão realizados testes de software, que atuará no topo da arquitetura (simulando a Camada de Aplicação) com o objetivo de gerar tráfego, forçar o protocolo e extrair métricas de desempenho. O fluxo deste módulo de teste automático consistirá nas seguintes etapas:

- I. Geração do payload: o emissor gera um array de dados com um padrão conhecido (por exemplo, uma sequência numérica sequencial de 68 bytes).
- II. Invocação da API: a módulo invoca a primitiva `sendFrame(payload, type)`, injetando a trama na Camada de Ligação.
- III. Transmissão e extração: a trama atravessa o sistema Full-Duplex (Camada 2 - Camada 1 - Canal Ótico - Camada 1 - Camada 2). Caso o campo FCS seja validado (no recetor), a API `receiveFrame(buffer)` é invocada para extrair o payload e entregá-lo ao módulo de teste para comparação.
- IV. Comparação: O módulo recetor efetua uma comparação direta e exata entre o payload extraído e o payload original.
- V. Avaliação Estatística: O processo é repetido num ciclo para um número elevado de iterações ($N = 100$ ou 1000 tramas).

Além disso, as métricas que serão extraídas, irão possibilitar conclusões quantitativas na apresentação final, tais como:

- Taxa de sucesso = percentagem de tramas que chegaram sem erros à primeira tentativa.
- Taxa de retransmissão: número de vezes que o temporizador (timeout) disparou ou que os mecanismos ARQ foram usados, para recuperar tramas corrompidas.
- FER (Frame Error Rate): a taxa de erro das tramas no canal (por exemplo, se o sistema regista uma falha a cada 100 tramas), permitindo comparar a viabilidade obtida com a capacidade teórica calculada.

4. Planeamento, Competências e Recursos

4.1 Competências Técnicas

Para cumprir os requisitos desta fase será necessário que o grupo adquira as seguintes competências:

- Configuração dos parâmetros de funcionamento da porta série;

- Capacidade de operar a porta série tanto no modo normal como no modo transparente (raw);
- Estruturação da trama, ou seja, fazer a divisão correta entre cabeçalho, payload e tail;
- Definição de diferentes tipos de tramas, para ser possível deteção e correção de erros e controlo de fluxo;
- Programação de algoritmos que permitam uma transmissão resistente aos mais diversos erros que poderão acontecer;
- Implementação de timeouts, para permitir ao sistema recuperar a transmissão em caso de erros serem detetados sem haver qualquer perda de tempo ou conteúdo;
- Aplicar o conceito de API na prática;
- Definição de funções dentro da API que permitam a comunicação entre o sistema físico da fase e o sistema da camada de ligação que será concebido nesta fase e futuramente com o sistema da camada aplicação que será projetado na fase C;
- Aplicar a lógica do protocolo de ligação no emissor e no recetor com o intuito de criar uma comunicação full-duplex;
- Definição de variáveis, tais como tamanho do ficheiro, tamanho máximo da trama e tempo de transmissão do ficheiro.

4.2 Planeamento de Tarefas

As tarefas foram organizadas de forma consecutiva no diagrama de Gant da **Figura 5**.

- Tarefa 1 - Definição Final de Estruturas e APIs:
 - Programar interface de comunicação (API) que fará a ponte entre a camada 2 e as restantes camadas, com funções que permitam abstrair a complexidade e com uma passagem de buffers otimizada para não sobrecarregar a memória do sistema.
 - Criar mecanismos de sincronização e delimitação nas tramas, que permite ao recetor recuperar e identificar novas tramas após perdas de sincronização ou erros no canal ótico.
 - Configurar a porta série das placas ESP32 para operar no modo raw, garantindo que os bytes da trama passam sem mudanças.
- Tarefa 2 - Implementação Base do Emissor e Recetor:

- Programar as funções de encapsulamento no emissor, de forma a receber o payload da Camada de Aplicação (Camada 7), inserir o cabeçalho (com endereços MAC e tipo), anexar o payload, e fechar com a cauda (FCS) para envio à Camada 1.
- Programar as funções de desencapsulamento no recetor que lê as tramas de forma contínua, aguardando por um SOF e assim lê, sucessivamente, o cabeçalho, o payload e a cauda, fazendo o mesmo com as próximas tramas, extraíndo o payload e dando-lhe print no terminal.
- Estabelecer a comunicação base full-duplex entre as duas placas ESP32 através do link ótico, garantindo a sua funcionalidade ao enviar tramas de ambos os PCs ao mesmo tempo.
- Tarefa 3 - Implementação do Controlo de Erros e Protocolo ARQ:
 - Programar o cálculo e verificação da Sequência de Verificação de Erros (FCS), no emissor calcula o CRC do cabeçalho e do payload e envia na trama, no recetor é recalculado e caso não corresponda ao esperado a trama é descartada.
 - Implementar a máquina de estados do protocolo Stop-and-Wait, com alternância de bits 0 e 1 para evitar o processamento de pacotes duplicados e garantindo o envio e receção de tramas de controlo (ACK).
 - Desenvolver a lógica de timeouts no emissor para forçar a retransmissão em caso de perda de pacotes ou de envio do ACK no momento errado, com um número máximo de transmissões pré-definido, que se for ultrapassado declara falha.
- Tarefa 4 - Simulação, Testes Práticos e Debugging:
 - Construir um módulo de software específico para testar a Camada de Ligação de forma isolada, gerando o envio de tramas de teste, para extrair estatísticas de sucesso, taxas de retransmissão e a taxa de erro de trama (FER).
 - Realizar as transferências práticas dos ficheiros hard coded (texto e imagem) sobre a porta série, caso sejam muitos longos devem ser divididos e enviados sequencialmente, com o recetor a acumulá-los até formar o ficheiro original.
 - Forçar erros no canal (interromper brevemente o canal ótico ou enviar um CRC errado de propósito) para validar se o sistema recupera a transmissão exatamente onde parou.

- Registrar os timestamps de início e fim da transferência, usando estes valores para calcular o rendimento e comparar com os cálculos teóricos de tempo total.
- Tarefa 5 - Revisão Final e Preparação da Demonstração:
 - Rever e comentar o código final.
 - Preparação para a apresentação.

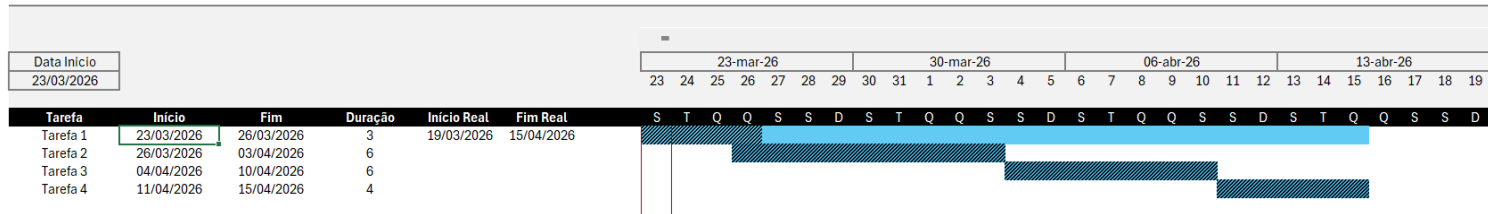


Figura 5 - Diagrama Temporal (Gant)

5. Conclusão

O relatório estabelece as especificações estruturais e funcionais para a fase B projeto, demonstrando a transição da camada física (Camada 1 do Modelo OSI) (Fase A) para a camada de ligação de dados (Camada 2 do Modelo OSI). A arquitetura que foi projetada tem como objetivo a implementação de um sistema de comunicação ponto a ponto, full-duplex e fiável entre dois computadores através de placas ESP32, permitindo o envio e receção de tramas. Para esse efeito, implementou-se uma estrutura de trama que inclui campos essenciais, como a delimitação da trama (SOF), o endereçamento MAC, a numeração de sequência e o controlo de erros (FCS). Sobre a fiabilidade do link ótico, justificou-se a utilização de algoritmos de deteção de erros como o Checksum e o CRC, e de mecanismos de correção de erros, como o ARQ. A escolha do ARQ é essencial, pois cumpre o requisito da recuperação da transmissão exatamente no ponto em que falhou, sendo isto possível através da utilização de temporizadores.

A base do controlo de fluxo e retransmissões assentará no protocolo *Stop-and-Wait*. A viabilidade desta abordagem foi comprovada através de cálculos de desempenho: a operar uma taxa de transmissão de 9600 bps com um payload de 64 bytes, estimou-se que a transferência de um ficheiro de teste de 1 KB seja realizada em aproximadamente 1,3 segundos. Este valor comprova as escolhas técnicas realizadas face às capacidades do hardware e às limitações do canal.

Resumindo, o relatório de especificação demonstra detalhadamente o que será realizado nesta fase do projeto, assegurando que os mecanismos de deteção e correção de

erros, e de controlo de fluxo são corretamente implementados, e que a transferência do ficheiro seja realizada de forma eficiente e rápida, assegurando que não haja corrupção ou perda de dados do ficheiro original.